

HelixCode

HelixCode

Распределённая платформа разработки AI, которая делит задачи, сохраняет прогресс и никогда не теряет ваше место.

Краткое описание

HelixCode — это корпоративная распределённая платформа разработки AI на базе Go, которая разбивает рабочие процессы на интеллектуальные задачи, распределяемые по сети управляемых SSH воркеров. Платформа автоматически сохраняет прогресс с помощью контрольных точек и отката, интегрирует несколько провайдеров LLM и поддерживает полный цикл разработки через интерфейсы REST, CLI, TUI и MCP.

Краткая аннотация

HelixCode — распределённая платформа разработки AI, написанная на Go. Она разбивает работу на интеллектуальные задачи, распределяемые по сети воркеров на базе SSH, сохраняет прогресс с помощью автоматических контрольных точек и отката, интегрирует несколько провайдеров LLM и обеспечивает полный цикл разработки через интерфейсы REST, CLI, TUI и MCP.

Подробное описание

HelixCode — это корпоративная распределённая платформа разработки AI (`dev.helix.code`, лицензия MIT), построенная на простом обещании, которое её слоган воплощает буквально: разделяй работу, сохраняй прогресс и никогда не теряй нить. Она создана для интеллектуального распределения задач, автоматического сохранения результатов и кроссплатформенных рабочих процессов, написана на Go с

учётом требований распределённых вычислений — поддержка многопоточности и портируемость в виде единого бинарного файла, а также автоматическое создание контрольных точек, откат и мониторинг в реальном времени реализованы как базовые примитивы, а не дополнительные модули.

Архитектура платформы включает поверхностный слой API на базе REST, WebSocket и MCP, под которым работают специализированные базовые сервисы: JWT для аутентификации и управления сессиями, пул воркеров на базе SSH с мониторингом состояния, управление задачами с контрольными точками и обработкой зависимостей, управление проектами и рабочими процессами, а также унифицированный слой провайдеров LLM. Всё это хранится на PostgreSQL, а Redis доступен как опциональный уровень координации и кэширования. Воркеры автоматически развёртываются в сети, поэтому масштабирование кластера сводится к указанию серверу нового устройства, а не к ручной настройке. Многоклиентские интерфейсы охватывают CLI, терминальный UI, REST и мобильные фреймворки, позволяя работать с платформой через скрипты, терминал или приложение.

HelixCode поддерживает полный цикл разработки от начала до конца: планирование, сборка, тестирование и рефакторинг выполняются автоматически с учётом зависимостей и контекста нескольких сессий, благодаря чему длительные процессы сохраняют целостность даже при прерываниях и смене устройств. Платформа интегрирует несколько провайдеров LLM — llama.cpp, Ollama и OpenAI — за единым интерфейсом, добавляет аппаратно-зависимый выбор моделей, определяя доступные CPU/GPU/память и подбирая модель под конкретное устройство. Также поддерживаются продвинутые стратегии рассуждений, такие как chain-of-thought и tree-of-thoughts, для задач, требующих более одного прохода. Model Context Protocol реализован на нескольких транспортных протоколах для стандартизированного обмена инструментами и контекстом, а многоканальные уведомления (Slack, Discord, Email, Telegram) держат команду в курсе прогресса распределённой работы. Платформа ориентирована на Linux, macOS, Windows, Aurora OS и SymphonyOS.

Почему мы его создали

Распределённая разработка с поддержкой AI обычно теряет контекст и прогресс, когда задачи распределяются между машинами или прерываются. HelixCode был создан, чтобы сделать разделение задач интеллектуальным, а сохранение работы — автоматическим. Благодаря этому крупные

разработки можно дробить, распределять по сети исполнителей, сохранять контрольные точки и возобновлять или откатывать без потери состояния.

Почему это меняет правила игры

Он делает распределённую разработку с AI *устойчивой* — возможности, которая раньше была недоступна, когда команды вручную собирали эти компоненты воедино. Три функции, обычно существующие в трёх отдельных инструментах, теперь объединены в одной платформе: распределённые вычисления (сети исполнителей SSH с автоустановкой и мониторингом работоспособности), поддержка разработки с AI (мультипровайдерные LLM с логическим выводом и вызовом инструментов) и полная автоматизация рабочих процессов на всём жизненном цикле. Связующим звеном выступают контрольные точки на базе базы данных: поскольку состояние задач, контрольные точки и зависимости сохраняются в PostgreSQL, работу, охватывающую множество машин и сессий, можно откатить или возобновить ровно с того места, где она была прервана. Прерывания и распределённая работа перестают быть источником потерь прогресса и становятся рядовым событием, из которого можно восстановиться.

Что здесь нового

- Сохранение работы как базовый примитив: автоматическое создание контрольных точек и откат применяются к *распределённым* задачам разработки, благодаря чему прогресс переживает прерывания и сбои оборудования, а не исчезает вместе с ними.
- Выбор модели с учётом аппаратных возможностей: система анализирует обнаруженные процессоры, GPU и объём памяти, подбирая для каждой задачи модель, которую машина способна эффективно выполнять, — без ручной настройки для каждого исполнителя.
- Одна платформа, пять точек входа: REST, WebSocket, CLI, TUI и MCP, причём MCP доступен через несколько транспортных протоколов, чтобы инструменты и агенты могли интегрироваться независимо от способа подключения.
- Кроссплатформенность, выходящая за рамки привычного трио настольных ОС: поддержка Aurora OS и SymphonyOS расширяет парк исполнителей за счёт платформ, которые большинство инструментов игнорируют.

Главные технические вызовы и их решения

- **Не терять работу при распределённых прерываемых задачах.** Когда задача дробится между машинами, любой сбой или прерывание обычно оставляют незавершёнными текущие процессы. Мы смоделировали саму задачу как носитель контрольных точек и зависимостей, сохраняемых в PostgreSQL, благодаря чему система может откатиться к последнему корректному состоянию или возобновить работу с него. Устойчивость обеспечивается на уровне данных, а не за счёт ненадёжного состояния в памяти.
- **Управление гетерогенным парком исполнителей.** Сеть машин под управлением Linux, macOS, Windows, Aurora и SymphonyOS — это постоянно меняющаяся среда с точки зрения доступности и настройки. Мы решаем эту проблему с помощью выделенного сервиса пула исполнителей, который выполняет регистрацию на базе SSH, автоустановку на новые узлы и непрерывный мониторинг работоспособности. Это позволяет поддерживать парк машин в управляемом состоянии, даже когда устройства появляются и исчезают.
- **Разнообразие провайдеров и аппаратных платформ.** Бэкенды LLM и машины, на которых они работают, сильно различаются по возможностям. Мы скрыли это за унифицированным интерфейсом провайдеров LLM и дополнили его аппаратным детектированием (процессор, GPU, память), которое обеспечивает интеллектуальный выбор модели. В результате нужная модель попадает на подходящую машину без необходимости для пользователя вникать в детали.

Технологический стек

- **Go (1.26+ с внутренним модулем)** — выбран за свою конкурентность на основе горутин и сборку в единый бинарный файл, что идеально подходит для распределённой системы воркеров: дешёвый параллелизм для оркестрации и самодостаточный бинарник, автоматически устанавливающийся на любой узел. Включает все основные сервисы и бинарные файлы CLI/server.
- **Gin (HTTP-фреймворк)** — выбран как быстрый и минималистичный слой REST с низкими накладными расходами; обслуживает поверхность /api/v1 (аутентификация, воркеры, задачи, проекты), с которой взаимодействует каждый клиент.
- **PostgreSQL 15+ (через pgx/v5)** — выбран как надёжная система хранения данных, поскольку контрольные точки и

откат требуют транзакционной устойчивости; содержит схему распределённых вычислений из 11 таблиц (пользователи, воркеры, задачи, проекты, сессии, провайдеры LLM, уведомления), обеспечивающую сохранность выполненной работы.

- **Redis 7+ (опционально, go-redis/v9)** — выбран как опциональный уровень кэширования и координации, ускоряющий горячие пути без превращения в жёсткую зависимость, так что минимальная установка может работать только на PostgreSQL.
- **SSH** — выбран как транспорт для управления воркерами именно потому, что он уже повсеместно распространён и защищён; обеспечивает регистрацию воркеров, автоматическую установку и удалённое выполнение команд по всему пулу без необходимости предварительного развёртывания специального агента.
- **Model Context Protocol (MCP)** — выбран для стандартизированного обмена инструментами и контекстом, чтобы внешние инструменты и агенты интегрировались через единый открытый протокол; реализован с поддержкой нескольких транспортов для взаимодействия с клиентами вне зависимости от способа подключения.
- **Провайдеры LLM (Llama.cpp, Ollama, OpenAI)** — выбраны для охвата как локального, так и облачного инференса за единым унифицированным интерфейсом, что позволяет маршрутизировать задачи на локальную модель или облачную без необходимости знать об этом вызывающей стороне.

Статус и замечания о честности

- **Статус: бета-версия.** В README заявлено состояние «ПОЛНОСТЬЮ ЗАВЕРШЕНО / все 5 этапов»; эта завершённость заявлена проектом, а не подтверждена независимой проверкой, поэтому страница рассматривает её как бета-версию.
- Все приведённые выше детали взяты из README репозитория; маркетинговые формулировки (слоганы) носят редакционный характер и не являются метриками из исходных данных.

Приоритетный уровень: Helix-primary.